

TASK 03 — JWT Access Token & Refresh Token Implementation

Step 1 — Study JWT

JWT stands for **JSON Web Token**.

It is a compact token used to securely represent information between a client and server.

A JWT has **3 parts**:

Header. Payload. Signature

Header

The header contains information about the token.

Example:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Meaning:

- alg → algorithm used to sign the token
- typ → token type

Payload

The payload contains **claims**, such as user information and token timing.

Example:

```
{  
  "sub": "john",  
  "role": "USER",  
  "iat": 1726650000,  
  "exp": 1726653600  
}
```

Here:

Claim Meaning

sub Subject/user

role User role

iat Token issued time

exp Token expiration time

JWT payload is encoded, not encrypted by default. Therefore, don't put passwords or other sensitive secrets inside it.

Signature

The signature helps the server verify that the token hasn't been modified.

Conceptually:

Signature =

HMAC(

Base64(Header) + "." + Base64(Payload),

Secret Key

)

If somebody changes the payload, the signature will no longer match.

Access Token

An **access token** is the JWT that the client sends to access protected APIs.

Example:

GET /api/products

Authorization: Bearer eyJhbGciOi...

The word:

Bearer

means that the client is presenting the token as proof of authentication.

Typical flow:

Login



Access Token



Client stores token



Client calls API



Authorization: Bearer <JWT>



Spring Security validates JWT



Allow / Reject request

Refresh Token

An **access token** should normally have a relatively short lifetime.

For example:

Access Token → 15 minutes

When it expires, the user doesn't necessarily need to log in again.

A **refresh token** can be used to obtain a new access token.

Example:

Login



Access Token → 15 min

Refresh Token → 7 days

After 15 minutes:

Refresh Token



New Access Token



Continue using API

A refresh token should be protected carefully because it can be used to obtain new access tokens.

Token Expiration

JWTs normally contain an expiration claim:

```
{  
  "sub": "john",  
  "exp": 1726653600  
}
```

After the expiration time:

JWT expired



Spring Security rejects request



HTTP 401 Unauthorized

This limits the amount of time a stolen access token can be used.

Stateless Authentication

This is an important concept in JWT authentication.

Traditional session authentication

The server stores a session:

Client



Login



Server creates Session



Session stored on Server

The server needs to remember the user's session.

JWT/stateless authentication

With JWT:

Client



Login



JWT generated



Client sends JWT with every request



Server validates JWT

The server doesn't need to maintain a login session for every client.

That's why JWT authentication is commonly described as **stateless authentication**.

Step 2 — Configure JWT

Add JWT dependencies

Open:

pom.xml

Add these dependencies:

```
<dependencies>

  <!-- Spring Web -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

  </dependency>

  <!-- Spring Security -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-security</artifactId>

  </dependency>

  <!-- JWT API -->

  <dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt-api</artifactId>

    <version>0.12.6</version>

  </dependency>

  <!-- JWT Implementation -->

  <dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt-impl</artifactId>

    <version>0.12.6</version>

    <scope>runtime</scope>

  </dependency>

  <!-- JWT Jackson -->

  <dependency>
```

```
<groupId>io.jsonwebtoken</groupId>  
<artifactId>jjwt-jackson</artifactId>  
<version>0.12.6</version>  
<scope>runtime</scope>  
</dependency>  
</dependencies>
```

Then reload Maven.

Create JWT configuration

Create this package: src/main/java/com/example/security/config

Create: JwtProperties.java

Code:

```
package com.example.security.config;  
  
import org.springframework.boot.context.properties.ConfigurationProperties;  
  
@ConfigurationProperties(prefix = "jwt")  
public class JwtProperties {  
    private String secret;  
  
    private long accessTokenExpiration;  
  
    private long refreshTokenExpiration;  
  
    public String getSecret() {  
        return secret;  
    }  
  
    public void setSecret(String secret) {  
        this.secret = secret;  
    }  
  
    public long getAccessTokenExpiration() {  
        return accessTokenExpiration;  
    }  
}
```

```

    }

    public void setAccessTokenExpiration(long accessTokenExpiration) {
        this.accessTokenExpiration = accessTokenExpiration;
    }

    public long getRefreshTokenExpiration() {
        return refreshTokenExpiration;
    }

    public void setRefreshTokenExpiration(long refreshTokenExpiration) {
        this.refreshTokenExpiration = refreshTokenExpiration;
    }
}

```

Step 3 — Generate Access Token

Login

The client sends the user's login details to your authentication API.

For example:

POST /api/auth/login

Request:

```

{
    "username": "john",
    "password": "password123"
}

```

The request goes to your Spring Boot application.

Validate credentials

Spring Security checks whether the username and password are correct.

Conceptually:

Username + Password



UserRepository



Find user



Check password



Correct?

/ \

YES NO



Continue 401

the database might contain:

username: john

password: <encoded password>

role: USER

The password should **not** be stored as plain text. It should be stored using a password encoder such as BCrypt.

Generate JWT

If the credentials are correct, your application creates a JWT.

Example:

User: john

Role: USER

Expiration: 15 minutes

These values can become JWT claims:

```
{
  "sub": "john",
  "role": "USER",
  "iat": 1726650000,
  "exp": 1726650900
}
```

The JWT is then signed using the secret key configured in **Step 2**.

So the relationship is:

Step 2

JWT Secret



Step 3

JWT Generation



Signed JWT

Return token

After generating the token, return it to the client.

A common response is:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer"
}
```

The client can then use this token for protected APIs.

For example:

GET /api/products

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Simple JwtService example

Create:

security/service/JwtService.java

```
package com.example.security.service;
```

```
import com.example.security.config.JwtProperties;
```

```
import io.jsonwebtoken.Jwts;
```

```
import io.jsonwebtoken.security.Keys;
```

```
import org.springframework.stereotype.Service;
```

```
import javax.crypto.SecretKey;
```

```
import java.nio.charset.StandardCharsets;
```

```
import java.util.Date;
```

```
@Service
```

```
public class JwtService {
```

```
    private final JwtProperties jwtProperties;
```

```
    public JwtService(JwtProperties jwtProperties) {
```

```
        this.jwtProperties = jwtProperties;
```

```
    }
```

```
    private SecretKey getSigningKey() {
```

```
        return Keys.hmacShaKeyFor(
```

```
            jwtProperties.getSecret()
```

```
                .getBytes(StandardCharsets.UTF_8)
```

```
        );
```

```
    }
```

```
    public String generateAccessToken(String username) {
```

```
        Date now = new Date();
```

```
        Date expiration = new Date(
```

```
        now.getTime()
        + jwtProperties.getAccessTokenExpiration()
    );
    return Jwts.builder()
        .subject(username)
        .issuedAt(now)
        .expiration(expiration)
        .signWith(getSigningKey())
        .compact();
    }
}
```

Step 4 — Add Claims

userId

Identifies which user the token belongs to.

Example:

```
"userId": 101
```

This means the authenticated user has ID 101.

email

Stores the user's email address.

```
"email": "user@gmail.com"
```

The server can identify the user's account from the token.

role

Stores the user's authorization role.

```
"role": "USER"
```

or:

```
"role": "ADMIN"
```

This is useful for **authorization**.

For example:

USER → Can view products

ADMIN → Can add/update/delete products

issuedAt

Specifies when the JWT was created.

Example:

```
"issuedAt": "2026-09-18T10:00:00"
```

It helps determine when the token was issued.

5. expiration

Specifies when the JWT becomes invalid.

Example:

```
"expiration": "2026-09-18T11:00:00"
```

After this time, the user needs to authenticate again or use a refresh-token mechanism.

Example JWT Payload

These claims can appear inside the JWT payload like this:

```
{  
  "userId": 101,  
  "email": "user@gmail.com",  
  "role": "USER",  
  "issuedAt": 1758199200,  
  "expiration": 1758202800  
}
```

The JWT is then signed by the server so that the server can detect unauthorized modification.

JWT is **not encrypted by default**. Its payload can generally be decoded, so **don't put passwords or other secrets inside claims**.

Step 5 — Create Authentication Response

accessToken

"accessToken": "JWT_TOKEN"

This is the JWT used to access protected APIs.

For example:

GET /api/products

Authorization: Bearer eyJhbGciOiJIUzI1Ni...

The client sends the access token with requests that require authentication.

refreshToken

"refreshToken": "REFRESH_TOKEN"

The refresh token is used to obtain a **new access token** when the access token expires.

For example:

Login



Access Token + Refresh Token



Access Token expires



Send Refresh Token



Receive new Access Token

tokenType

"tokenType": "Bearer"

Bearer tells the client how the access token should be sent.

The HTTP request normally looks like:

Authorization: Bearer <access-token>

expiresIn

"expiresIn": 3600

This indicates how long the access token is valid, usually in **seconds**.

3600 seconds



60 minutes



1 hour

So the example access token expires after **1 hour**.

Step 6 — Create JWT Authentication Filter

Read Authorization Header

The client sends the JWT in the HTTP Authorization header.

Authorization: Bearer eyJhbGciOiJIUzI1Ni...

The filter reads this header.

Extract Bearer Token

The filter checks whether the header starts with:

Bearer

Then it extracts only the token.

For example:

Authorization:

Bearer eyJhbGciOiJIUzI1Ni...

becomes:

eyJhbGciOiJIUzI1Ni...

Validate JWT

The filter verifies the token.

It checks things such as:

- Is the token correctly signed
- Is the token expired
- Has the token been modified
- Is the token structurally valid

If validation fails, the request should not be treated as authenticated.

Extract User Information

If the JWT is valid, the filter reads information from its claims.

For example:

```
{  
  "userId": 101,  
  "email": "user@gmail.com",  
  "role": "USER"  
}
```

The application can use this information to identify the authenticated user and their role.

Set SecurityContext

This is an important Spring Security step.

The filter creates an authenticated Authentication object and puts it into:

SecurityContextHolder

Conceptually:

Valid JWT



User information



Authentication object



SecurityContext

Spring Security can then understand:

"This request is coming from an authenticated user."

Your controller can subsequently use Spring Security features such as:

```
@PreAuthorize("hasRole('ADMIN')")
```

Step 7 — Secure APIs

Once JWT authentication is configured, you can protect your API endpoints.

For example:

GET /api/products

The client sends:

Authorization: Bearer <token>

Successful request

Client

↓

GET /api/products

Authorization: Bearer JWT

↓

JWT Filter

↓

Token valid

↓

SecurityContext created

↓

ProductController

↓

200 OK

Without a valid token

Client



GET /api/products



JWT Filter



No valid authentication



Request rejected

Typically, a protected endpoint returns **401 Unauthorized** when authentication is missing or invalid.

Step 8 — Test Invalid Tokens

Missing Token

Request:

GET /api/products

There is no:

Authorization: Bearer <token>

For a protected endpoint, the request should be rejected.

Invalid Token

Example:

Authorization: Bearer abc123invalid

The token isn't a valid JWT or doesn't have a valid signature.

Expected result:

401 Unauthorized

Expired Token

Suppose the token contains:

```
{  
  "expiration": 1758202800  
}
```

and that expiration time has passed.

The JWT should no longer be accepted.

The client needs to use the appropriate refresh-token flow or authenticate again, depending on your application's design.

Modified Token

Suppose the original token contains:

```
{  
  "userId": 101,  
  "role": "USER"  
}
```

Someone modifies the payload to:

```
{  
  "userId": 101,  
  "role": "ADMIN"  
}
```

If the JWT is properly signed, the signature will no longer match the modified contents.

Modified JWT



Signature validation fails



Authentication fails



Request rejected

This is one of the main reasons **JWT signing** is important.

Step 9 — Implement Refresh Token

Usually, the access token should have a limited lifetime.

For example:

Access Token → Valid for 1 hour

Refresh Token → Valid for a longer period

After one hour:

Access Token expired



Client sends Refresh Token



Server validates Refresh Token



Server generates new Access Token



Client can continue using the API

So the user doesn't need to enter their username and password again.

Login

First, the user logs in:

POST /api/v1/auth/login

Request:

```
{  
  "email": "user@gmail.com",  
  "password": "password123"  
}
```

After successful authentication, the server returns:

```
{
```

```
"accessToken": "eyJhbGciOiJIUzI1Ni...",  
"refreshToken": "abc123xyz...",  
"tokenType": "Bearer",  
"expiresIn": 3600  
}
```

Use Access Token

The client uses the access token to call a protected API:

GET /api/v1/products

Authorization: Bearer eyJhbGciOiJIUzI1Ni...

The JWT authentication filter validates the token.

If valid:

Request → JWT Filter → SecurityContext → Controller

Access Token Expires

Suppose:

Access Token lifetime = 3600 seconds

That is:

3600 seconds = 60 minutes = 1 hour

After one hour, the access token is no longer valid.

The client should **not keep sending the expired access token**.

Instead, it calls:

POST /api/v1/auth/refresh

4. Send Refresh Token

Example request:

```
{  
  "refreshToken": "abc123xyz..."  
}
```

The server receives the refresh token and validates it.

It should verify things such as:

- ✓ Refresh token exists
- ✓ Refresh token is valid
- ✓ Refresh token has not expired
- ✓ Refresh token belongs to the expected user/session
- ✓ Refresh token has not been revoked

Generate New Access Token

If the refresh token is valid:

Refresh Token



Validate



Valid?



YES



Generate New Access Token

Example response:

```
{  
  "accessToken": "NEW_JWT_TOKEN",  
  "refreshToken": "abc123xyz...",  
  "tokenType": "Bearer",  
  "expiresIn": 3600  
}
```

Depending on your design, you may also **rotate the refresh token**, meaning the server issues a new refresh token and invalidates the old one.

Access Token vs Refresh Token

Feature	Access Token	Refresh Token
Purpose	Access protected APIs	Get a new access token
Lifetime	Short	Longer
Sent with normal API requests	Yes	No
Example	JWT	Random/opaque token or JWT
Used by	API authorization	Token refresh endpoint

A refresh token should generally be protected more carefully than an access token because possession of a valid refresh token can allow continued issuance of access tokens.